**Listing 7.3 `chatgpt_clone_response.py` (top section)**

```
import streamlit as st
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()      #1

st.title("ChatGPT-like clone")

client = OpenAI()      #2

if "openai_model" not in st.session_state:
    st.session_state["openai_model"]
              = "gpt-4-1106-preview"     #3

if "messages" not in st.session_state:
    st.session_state["messages"] = []   #4

for message in st.session_state["messages"]:      #5
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
```

**#1 Loads the environment variables from the .env file**
**#2 Configures the OpenAI client**
**#3 Checks the internal session state for the setting, and adds it if not there**
**#4 Checks for the presence of the message state; if none, adds an empty list**
**#5 Loops through messages in the state and displays them**

The Streamlit app itself is stateless. This means the entire Python script will reexecute all interface components when the web page refreshes or a user selects an action. The Streamlit state allows for a temporary storage mechanism. Of course, a database needs to support more long-term storage.

UI controls and components are added by using the `st.` prefix and then the element name. Streamlit supports several standard UI controls and supports images, video, sound, and, of course, chat.

Scrolling down further will yield listing 7.4, which has a slightly more complex layout of the components. The main `if` statement controls the running of the remaining code. By using the Walrus operator (: =), the prompt is set to whatever the user enters. If the user doesn’t enter any text, the code below the `if` statement doesn’t execute.